Pineapple-Bot-Pro

A Low-Cost Mecanum-Drive Robot with Latency-Compensated Vision and Jerk-Limited Trajectory Control

Technical Report

Vietnam Robotics For Good — National Competition

Hosted at USTH (Đại Học Việt Pháp)

Team Pineapple

April 2026

Contents

1 Abstract

This report presents the design and implementation of the **Pineapple-Bot-Pro**, an autonomous mecanum-wheeled robot system developed for the *Vietnam Robotics For Good* national competition. The system is built on a deliberately constrained, low-cost hardware platform centered on an ESP32-S3 microcontroller, commodity brushed motors, and Tower Pro SG90 micro-servos, demonstrating that sophisticated autonomous behavior can be achieved without expensive sensors or actuators.

The primary contribution of this work is a tightly integrated software architecture spanning two distinct platforms—an ESP32-S3 embedded firmware and an Android companion application—that together implement several novel techniques for overcoming the limitations of low-cost hardware. These include: (1) an *asymmetric dual-core firmware architecture* that dedicates one processor core to a 1 kHz motor control loop while the other handles network communication; (2) a *jerk-limited S-curve trajectory profiler* with velocity and acceleration feedforward compensation; (3) a *latency-compensated vision system* that uses historical dead-reckoning state to retroactively correct for the ∼150 ms camera-to-actuation delay inherent in smartphone-based tracking; (4) a *single-motor micro-adjustment mode* that exploits mecanum kinematics to achieve sub-2 mm positional corrections; and (5) a *physics-based predictive braking system* with an adaptive lookahead that scales with robot velocity.

The system achieves reliable waypoint navigation with ±5 mm positional tolerance and ±1.5° angular tolerance on a 1143 mm × 1181 mm competition field, using only a consumer smartphone camera for external localization.

2 Introduction

2.1 Competition Context

The *Vietnam Robotics For Good* competition challenges teams to design autonomous robotic systems capable of performing precision manipulation tasks on a structured playing field. The competition emphasizes practical engineering under real-world constraints: limited budgets, unreliable network conditions, and the demand for robust autonomous operation without human intervention during runs.

The Pineapple-Bot-Pro system was developed under a deliberate philosophy of *cost-constrained sophistication*. Rather than investing in high-end hardware—industrial servo motors, LiDAR sensors, or dedicated vision processors—the team elected to maximize the algorithmic and architectural complexity of the software stack running on commodity components. This approach demonstrates that careful systems engineering can compensate for hardware limitations, a principle with broad applicability beyond competitive robotics.

2.2 System Overview

The Pineapple-Bot-Pro system comprises two tightly coupled subsystems:

1. **Robot Firmware**: A PlatformIO-based C++ application running on an ESP32-S3 microcontroller. The firmware implements a dual-core architecture with dedicated tasks for motor control (1 kHz on Core 1) and network communication (Core 0). Key

subsystems include a 16-segment motion queue with S-curve trajectory profiling, a Kalman-filtered dead-reckoning engine with camera-based latency compensation, and a four-motor mecanum drive controller with multiple operating modes.

2. **Android Controller Application**: A Kotlin/Jetpack Compose application that serves as both the external vision system and the mission sequencer. The app uses the smartphone's rear camera with OpenCV-based ArUco marker detection to provide real-time position and orientation observations of the robot on the playing field. It implements a graph-based path editor, a continuous motion controller with waypoint sequencing, and a structured UDP communication bridge for robot telemetry and command dispatch.

?? illustrates the high-level architecture.

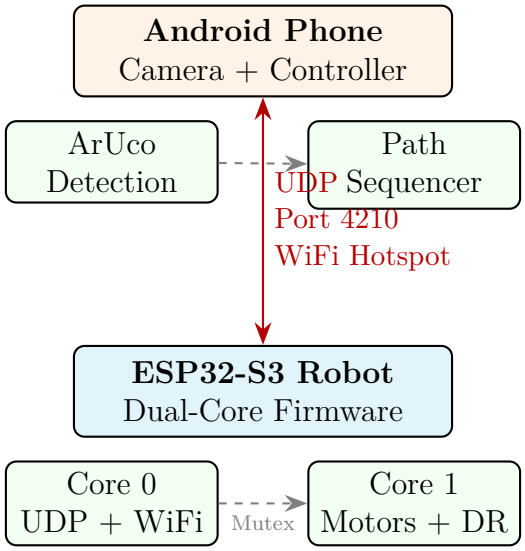

Figure 1: High-level system architecture. The Android phone acts as both the external vision sensor and the mission controller, communicating with the robot over a direct WiFi hotspot link via UDP on port 4210.

2.3 Design Philosophy

Several guiding principles shaped the system design:

- **Software Over Hardware:** Invest in algorithmic sophistication rather than expensive components. A $3 ESP32-S3 can execute a 1 kHz control loop with S-curve profiling, Kalman filtering, and predictive braking—capabilities typically associated with dedicated motion controllers costing orders of magnitude more.

- **Graceful Degradation:** The system must remain functional even when individual subsystems fail. If camera observations stop arriving, the robot continues on dead reckoning. If network latency spikes, the latency-aware speed governor prevents overshoot.

- **Multi-Robot Generality:** The firmware supports a layered configuration system that allows the same codebase to run on physically distinct robots with different calibration parameters, mass distributions, and servo configurations.

3 Hardware Platform

3.1 Microcontroller

The robot is powered by the **Espressif ESP32-S3-DevKitM-1**, a dual-core Xtensa LX7 processor clocked at 240 MHz. The ESP32-S3 was selected for its combination of dual-core processing capability, integrated WiFi (802.11 b/g/n), generous SRAM (512 kB), and extremely low unit cost (~$3 USD). The Arduino framework is used via PlatformIO for development convenience, with FreeRTOS primitives (mutexes, task pinning) employed for explicit dual-core scheduling.

Key configuration parameters from `platformio.ini`:

- CPU frequency: 240 MHz
- Framework: Arduino (ESP-IDF underneath)
- Serial monitor: 921 600 baud
- USB mode: CDC-on-boot for seamless serial debugging

3.2 Drive System

The robot employs a **four-wheel mecanum drive** using brushed DC motors (speculative model: ST130-12240-38 or equivalent). Mecanum wheels were chosen for their holonomic mobility—the ability to translate in any direction without rotating the chassis—which is essential for the precision positioning tasks required by the competition.

The motors are driven via a PWM-based H-bridge controller library (`ESP32_Motor_Controller`), with duty cycles ranging from 0 to 100. The wheel layout follows a standard mecanum configuration:

Table 1: Motor assignment and layout.

Motor	Position	Side	Balance Offset
M1	Front Right	Right	−3.0 duty
M2	Front Left	Left	—
M3	Back Left	Left	—
M4	Back Right	Right	−3.0 duty

Motors M1 and M2 are configured with reversed polarity in firmware to account for opposite physical mounting orientations.

3.3 Servo Actuators

The robot uses up to four **Tower Pro SG90** micro-servos for manipulation tasks. The SG90 is an ultra-low-cost (~$1 USD) hobby servo with the following nominal specifications:

Table 2: Tower Pro SG90 servo specifications (nominal).

Parameter	**Value**
Operating voltage	4.8 V to 6.0 V
Stall torque	1.8 kg · cm at 4.8 V
Operating speed	0.1 s / 60°
Rotation range	180°
Weight	9 g

The firmware supports two distinct servo configurations, conditionally compiled for each robot:

- **Robot A (Fruit Picker):** Three servos configured as a grabber (left/right/center sweep), a vertical slider (up/down), and an arm (up/down). Combined, these enable compound pick-and-place sequences such as `LOWER_LEFT`, `UPPER_RIGHT`, etc.
- **Robot B (Dispenser):** Two servos configured as a dropper (open/close) and a ceiling mechanism (open/close/water).

3.4 Communication Hardware

Rather than employing a dedicated radio link (e.g., Bluetooth, LoRa, or XBee), the system uses the ESP32-S3's **integrated WiFi** to connect to an Android phone configured as a mobile hotspot. This eliminates the need for additional transceiver hardware and leverages the smartphone's existing networking stack. The phone's gateway IP is automatically used as the command endpoint, requiring zero manual network configuration.

3.5 External Vision System

The external localization system is the **smartphone's rear camera**, processed via CameraX with OpenCV. This deliberately eschews dedicated vision hardware (e.g., Intel RealSense, overhead cameras with dedicated PCs) in favour of a single consumer device that also serves as the mission controller, minimizing both cost and physical setup complexity.

3.6 Physical Specifications

Two robot configurations are supported, with the key physical parameters summarized in ??.

Table 3: Physical specifications of Robot A and Robot B.

Parameter	**Robot A**	**Robot B**
Mass	460 g	926.3 g
Actuation force (vertical)	3.79 N	Variable
Static friction coefficient	0.847	0.372
Traction limit	8308 mm/s^2	3652 mm/s^2
Motor torque limit	8242 mm/s^2	—
Dominant constraint	Motor-limited	Traction-limited

The kinematic implications of these measurements are significant: Robot A is *traction-dominant*—the motors physically cannot output enough torque to break traction, making it inherently immune to acceleration-induced slip. Robot B, conversely, is *traction-limited*, requiring firmware-enforced acceleration caps to prevent wheel slip and ensure dead-reckoning accuracy.

4 Firmware Architecture

4.1 Asymmetric Dual-Core Design

The ESP32-S3's dual Xtensa LX7 cores are partitioned asymmetrically:

- **Core 0 (Network & Parsing):** Runs the `udpTask` as a FreeRTOS task at priority 2. This core handles WiFi stack events, asynchronous UDP packet reception and parsing, and periodic telemetry broadcasting (STATUS at 10 Hz, HELLO at 0.33 Hz, PING at 2 Hz).

- **Core 1 (Motion & Kinematics):** Runs the Arduino `loop()` function, dedicated to the high-frequency motor control loop targeting 1 kHz (1 ms interval). This core processes the motion queue, computes trajectory profiles, updates dead reckoning, and drives the motors.

State synchronization between cores is managed via a single FreeRTOS mutex (`stateMutex`) that guards access to the shared motion queue, dead reckoning engine, and latency compensator. The ABORT command is given special priority: Core 0 sets a volatile `abortFlag` that Core 1 checks at the top of every control loop iteration, enabling sub-millisecond emergency stop response without waiting for the mutex.

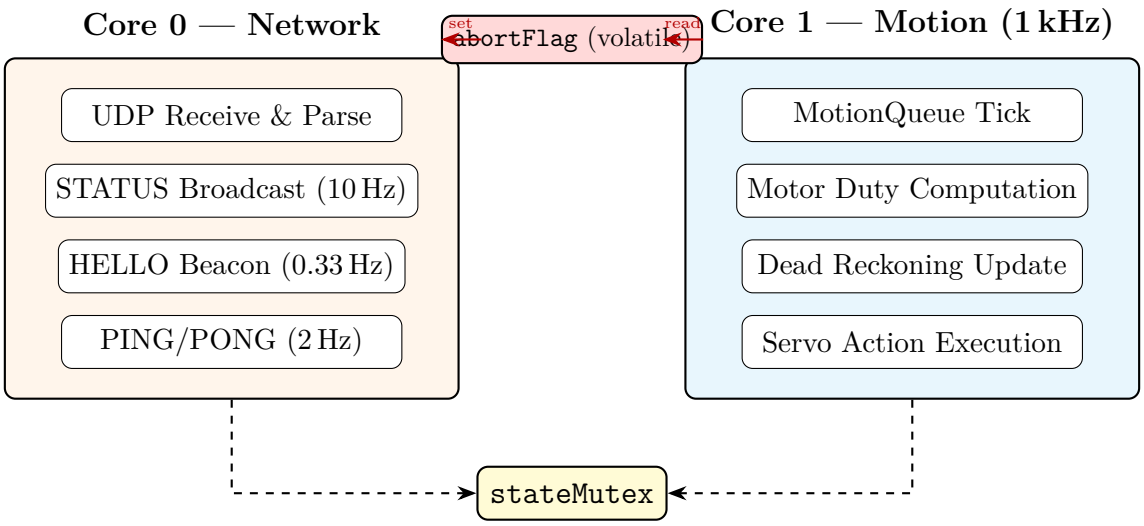

Figure 2: Dual-core task partitioning. Core 0 handles all network I/O while Core 1 runs the deterministic 1 kHz control loop. Shared state is guarded by a FreeRTOS mutex; the emergency abort flag bypasses the mutex for minimal latency.

4.2 Multi-Robot Configuration System

The firmware employs a **three-layer configuration hierarchy** that enables the same codebase to run on physically distinct robots:

1. **`config_defaults.h`**: Contains every tunable constant with a safe baseline value, using `#ifndef`/`#define` guards.
2. **`config_a.h`/`config_b.h`**: Robot-specific overrides that `#include` the defaults first, then redefine only the parameters that differ (e.g., distance calibration factors, drift trim angles, WiFi credentials).
3. **`main.h`**: Selects the active robot configuration via a single `#include` directive and declares the shared API.

This layered approach ensures that adding a new robot (e.g., Robot C) requires only copying an existing config file and adjusting the calibrated values—no changes to the core firmware logic.

4.3 Subsystem Library Architecture

The firmware is decomposed into six modular libraries, each with a well-defined interface:

Table 4: Firmware library decomposition.

Library	Responsibility	Size
`MotionQueue`	Motion segment queueing, S-curve profiling, predictive braking, Kalman velocity estimation, anti-slip detection	∼74 kB
`DeadReckoning`	Odometric position integration, history ring buffer, anchor-based correction model	∼11 kB
`LatencyCompensator`	Camera latency compensation, RTT tracking, clock synchronization, drift correction	∼14 kB
`UdpProtocol`	Message parsing (ASCII), message building, enum definitions	∼17 kB
`ServoControl`	Servo action sequences, per-robot conditional compilation	∼5 kB
`UdpLogger`	Structured remote logging with static, update-in-place, and important message types	—

5 Communication Protocol

5.1 Design Rationale

The robot–phone communication uses a custom **UDP protocol** with short ASCII strings. This design prioritizes:

- **Minimal parsing overhead:** Simple colon-delimited fields avoid the complexity of binary serialization or protocol buffer libraries, which are challenging to integrate on embedded platforms with limited memory.

- **Human readability:** Messages are directly readable on serial monitors and in log files, dramatically simplifying debugging during development and competition.

- **Tolerance of packet loss:** UDP's lack of retransmission is acceptable because the system is designed around continuous state updates rather than reliable delivery. A lost STATUS message is superseded by the next one 100 ms later; a lost WAYPOINT command is re-sent by the phone's control loop.

All communication occurs on a single UDP port (**4210**), with the phone acting as a WiFi hotspot and the robot connecting as a station. The phone's gateway IP is automatically detected by the robot at startup.

5.2 Protocol Messages

?? summarizes the complete message set.

Table 5: UDP protocol message reference. Direction: ⇒ = Phone to Robot, ⇐ = Robot to Phone, ⇔ = bidirectional.

Prefix	Name	Dir.	Format & Description
`H`	HELLO	⇐	`H:{id}:{ip}` — Auto-discovery beacon, every 3 s
`R`	REGISTER	⇐	`R:{id}:mecanum4wd:{ip}` — Capability registration
`S`	STATUS	⇐	`S:{y}:{x}:{angle}:{qLen}:{drift}` — Telemetry at 10 Hz
`L`	LOG	⇐	`L:{S|U|I}[:{id}]:{msg}` — Remote debug logging
`P`	PING	⇔	`P:{timestamp}` — RTT measurement
`Q`	PONG	⇔	`Q:{orig_ts}:{remote_ts}` — Clock synchronization
`M`	MOVE	⇒	`M:{dir}:{dist}:{speed}:{policy}[:{action}]`
`D`	MOVE_DUR	⇒	`D:{dir}:{dur_ms}:{speed}:{policy}`
`W`	WAYPOINT	⇒	`W:{x}:{y}[:{angle}]:{speed}:{policy}[:{action}]`
`T`	ROTATE	⇒	`T:{target_angle}:{speed}:{policy}`
`O`	ROT_DUR	⇒	`O:{cw|ccw}:{dur_ms}:{speed}:{policy}`
`V`	VELOCITY	⇒	`V:{vx}:{vy}:{omega}:{timeout_ms}`
`C`	CAM	⇒	`C:{ts}:{x}:{y}:{angle}` — Camera observation
`E`	SERVO	⇒	`E:{action_code}` — Standalone servo execution
`U`	SERIAL	⇒	`U:{0|1}` — Enable/disable serial log streaming
`X`	DONE	⇒	Mission completion signal (triggers LED feedback)
`A`	ABORT	⇒	Immediate emergency stop

5.3 Coordinate System Convention

A notable protocol detail is the **axis swap** in STATUS messages: the robot transmits `S:{y}:{x}:{angle}:{qLen}:{drift}`, with the x and y coordinates transposed relative to the robot's internal frame. This compensates for the 90° rotation between the phone's camera coordinate system (where the phone is held in landscape, looking down at the field) and the robot's internal navigation frame, without requiring coordinate transforms in the latency-critical control loop.

5.4 Waypoint Deduplication

The WAYPOINT command includes a **deduplication filter**: repeated commands within 15 mm tolerance of the currently active target are silently ignored. This prevents the phone's camera-triggered control loop from flooding the robot with redundant waypoint commands that would each trigger a queue abort-and-re-enqueue cycle, causing motion stuttering.

5.5 Remote Logging System

The firmware implements a structured **remote logging system** that streams diagnostic data to the phone when enabled via the `U` command. Three log types are supported:

- **Static (S):** One-off log entries appended to a scrolling log buffer.
- **Update-in-place (U):** High-frequency telemetry identified by a string key, overwriting the previous value—used for PWM values, PID errors, and motor diagnostics without flooding the log.
- **Important (I):** Highlighted entries rendered in red on the phone, used for critical state changes and errors.

This three-tier system enables real-time debugging during competition runs without overwhelming the display with noise from high-frequency diagnostic channels.

6 Motion Control and Trajectory Planning

The motion control subsystem is the most algorithmically sophisticated component of the firmware, converting high-level navigation intents into smooth, precise motor actuation.

6.1 Motion Queue Architecture

The **MotionQueue** subsystem implements a FIFO queue of up to 16 `MotionSegment` objects. Each segment encodes a complete motion intent: target position, velocity vector, speed level, correction policy, and optional servo action to execute upon arrival.

The queue supports several segment types:

- **Distance-based moves** (`MOVE`): Move in a cardinal direction for a specified distance.
- **Duration-based moves** (`MOVE_DUR`, `ROT_DUR`): Move or rotate for a specified time.
- **Waypoint navigation** (`WAYPOINT`): Navigate to absolute coordinates (x, y) with a target heading.
- **Rotation** (`ROTATE`): Turn to an absolute angle.
- **Direct velocity** (`VELOCITY`): Apply world-frame velocity (v_x, v_y, ω) with a timeout.

Segments execute back-to-back without stops, enabling fluid multi-step sequences.

6.2 Coasting Bridge

A persistent challenge with queued motion segments is **inter-segment stuttering**: when the queue momentarily empties between contiguous commands due to network jitter, the robot briefly stops and restarts, producing jerky motion. The firmware addresses this with a **Coasting Bridge**—a 40 ms hold window (`motionCommandGapHoldMs`) that sustains the previous velocity if the queue empties between segments. This bridges the gap until the next command arrives, maintaining smooth continuous motion.

6.3 S-Curve Trajectory Profiling

Standard trapezoidal velocity profiles produce instantaneous changes in acceleration (infinite jerk), which excite mechanical resonances and cause wheel slip on low-traction surfaces. The Pineapple-Bot-Pro employs a **jerk-limited S-curve profiler** that constrains not only velocity and acceleration but also the *rate of change of acceleration* (jerk).

The S-curve profiler maintains an internal state consisting of the current acceleration $a(t)$ and profiled speed $v(t)$. At each control tick (interval Δt), it advances toward the target speed v_{target} as follows:

1. Compute the desired acceleration: $a_{\text{desired}} = \text{clamp}\left(\frac{v_{\text{target}} - v(t)}{\Delta t},\ -a_{\max},\ a_{\max}\right)$
2. Apply jerk limiting: $\Delta a = \text{clamp}(a_{\text{desired}} - a(t),\ -j_{\max} \cdot \Delta t,\ j_{\max} \cdot \Delta t)$
3. Update state: $a(t + \Delta t) = a(t) + \Delta a, \quad v(t + \Delta t) = v(t) + a(t + \Delta t) \cdot \Delta t$

The profiler uses **asymmetric limits** for acceleration and deceleration:

Table 6: S-curve profiler parameters.

Parameter	Acceleration	Deceleration
Peak acceleration (linear)	$250\,\text{mm/s}^2$	$500\,\text{mm/s}^2$
Peak jerk (linear)	$1200\,\text{mm/s}^3$	$3000\,\text{mm/s}^3$
Peak acceleration (angular)	$180\,\text{deg/s}^2$	$360\,\text{deg/s}^2$
Peak jerk (angular)	$900\,\text{deg/s}^3$	$2250\,\text{deg/s}^3$

The asymmetry—deceleration limits are 2× the acceleration limits, and deceleration jerk is 2.5×—allows the robot to brake more aggressively than it accelerates, which is physically appropriate (braking adds friction rather than fighting it) and improves stopping precision.

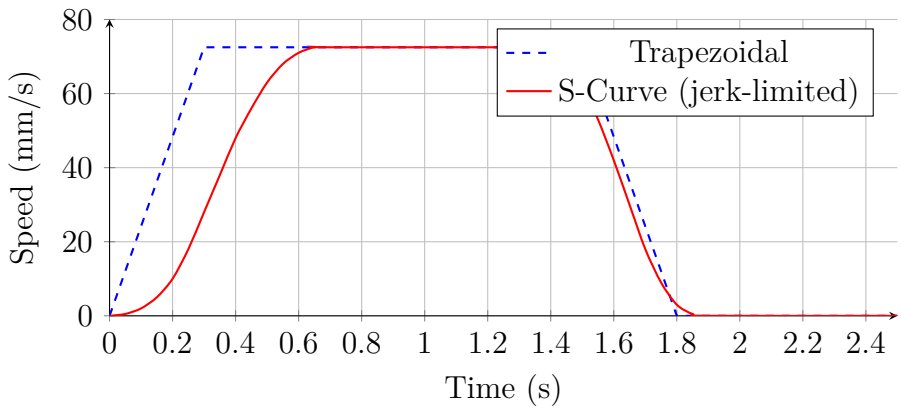

Figure 3: Comparison of trapezoidal and S-curve velocity profiles. The S-curve profile eliminates the discontinuities in acceleration that cause wheel slip and mechanical vibration on low-cost hardware.

6.4 Velocity and Acceleration Feedforward

The S-curve profiler generates not only the target velocity but also the current acceleration, which is used for **feedforward compensation**. The feedforward terms pre-compensate for inertia and friction, reducing tracking lag in the closed-loop controller:

$$v_{\text{ff}} = K_v \cdot v(t) + K_a \cdot a(t) \tag{1}$$

where $K_v = 0.02$ and $K_a = 0.04$ are empirically tuned gains. The feedforward is computed in world frame by the motion queue but must be projected into the robot's **heading frame** using the inverse rotation transform:

$$V_{\text{ff}} = \frac{v_{\text{ff},x} \cos\theta - v_{\text{ff},y} \sin\theta}{v_{\max}} \tag{2}$$

$$H_{\text{ff}} = \frac{v_{\text{ff},x} \sin\theta + v_{\text{ff},y} \cos\theta}{v_{\max}} \tag{3}$$

Note the use of the *inverse* (transpose) rotation matrix rather than the forward rotation. This subtlety is critical: the feedforward compensates for world-frame inertia, which must be projected *into* the robot frame for correct motor drive computation.

6.5 Adaptive Lookahead

The steering algorithm uses an **adaptive lookahead** that scales with the robot's current speed:

$$t_{\text{lookahead}} = \text{clamp}(t_{\text{base}} + v \cdot k_{\text{gain}},\ t_{\min},\ t_{\max}) \tag{4}$$

with $t_{\text{base}} = 0.06\,\text{s}$, $k_{\text{gain}} = 0.0022$, $t_{\min} = 0.04\,\text{s}$, and $t_{\max} = 0.18\,\text{s}$.

At high speeds, the extended lookahead accounts for greater momentum and prevents oscillatory tracking. At low speeds near the target, the shortened lookahead provides tighter control for precision positioning. This is substantially more effective than a fixed lookahead, which must compromise between stability at speed and responsiveness at rest.

6.6 Predictive Braking

The robot implements a **physics-based overshoot prevention** mechanism. At each control tick, it computes the maximum safe entry speed for the remaining distance assuming a fixed deceleration capability:

$$v_{\text{safe}} = \sqrt{2 \cdot a_{\text{decel}} \cdot d_{\text{remaining}}} \tag{5}$$

where $a_{\text{decel}} = 350\,\text{mm/s}^2$ and a $1.2\times$ safety margin is applied. If the current closing speed exceeds v_{safe}, the velocity scale is proactively reduced. This prevents the common failure mode where a robot approaches a waypoint too fast, overshoots, turns around, overshoots again, and oscillates indefinitely.

6.7 Latency-Aware Speed Governor

A complementary mechanism addresses the specific problem of **network latency**: when the phone sends a "stop" or "new waypoint" command, the robot may not receive it for $\sim$150 ms. During this window, the robot continues at its current speed and may overshoot. The latency-aware governor imposes a hard upper bound on commanded speed:

$$v_{\text{cap}} = \sqrt{2 \cdot a_{\text{latency}} \cdot d_{\text{remaining}}} \tag{6}$$

where $a_{\text{latency}} = 300\,\text{mm/s}^2$ (more conservative than the predictive brake). This ensures the robot can always stop within the remaining distance, even if the next command is delayed by a full network round-trip.

6.8 Blended Angle Stabilization

During waypoint navigation, the robot must maintain a target heading while translating. A naïve approach would use either the internal dead-reckoning angle (which drifts) or the camera ground-truth angle (which arrives sporadically and can jump). Either choice causes problems: DR drift leads to systematic heading error, while camera jumps translate into "strafe-jumps" where the heading-frame velocity transform suddenly shifts the motor outputs.

The Pineapple-Bot-Pro solves this with a **Blended Angle**: an EMA (Exponential Moving Average) fusion of the high-frequency DR angle with the low-frequency camera ground-truth angle. This provides a heading reference that is:

- **Smooth**: The EMA prevents discontinuities from camera observations.
- **Accurate**: The blend converges toward ground truth over time.
- **Responsive**: The high-frequency DR component tracks fast rotations between camera updates.

The blended angle is used for both the PID heading stabilization loop and the heading-frame velocity transform, ensuring that corrections from the tracker do not translate into jerky lateral movements.

7 Motor Kinematics and Actuation

7.1 Mecanum Drive Model

The four-motor mecanum drive converts world-frame velocity commands (v_x, v_y, ω) into individual motor duties. The kinematic model first transforms world-frame velocities into the robot's heading frame using the blended angle θ:

$$V = \frac{v_x \cos\theta + v_y \sin\theta}{v_{\max}} \qquad \text{(forward/backward)} \tag{7}$$

$$H = \frac{v_x \sin\theta - v_y \cos\theta}{v_{\max}} \qquad \text{(strafe left/right)} \tag{8}$$

The H-axis output is scaled by $1/\sqrt{2} \approx 0.707$ to normalize the mecanum geometry, which naturally produces $\sqrt{2}\times$ faster strafe than forward motion. The individual motor speeds follow the standard mecanum formula:

$$m_1 = -H - V - A \qquad \text{(Front Right)} \tag{9}$$

$$m_2 = -H + V - A \qquad \text{Front Left} \tag{10}$$

$$m_3 = -H + V + A \qquad \text{(Back Left)} \tag{11}$$

$$m_4 = -H - V + A \qquad \text{(Back Right)} \tag{12}$$

where $A = -\omega/\omega_{\max}$ is the normalized angular velocity.

7.2 Dead Zone Compensation and Linear Calibration

Low-cost brushed motors exhibit a significant **dead zone**—a range of duty cycles too low to overcome static friction. The firmware bypasses this nonlinearity with a calibrated linear mapping:

$$\text{duty} = 43.017 \cdot |\text{speed}| + 57.165 \cdot \text{sign}(\text{speed}) \tag{13}$$

This mapping was determined empirically by plotting commanded duty vs. measured wheel velocity, fitting a linear regression, and extracting the slope (43.017) and offset (57.165). The offset term ensures that any nonzero speed command immediately jumps past the dead zone.

7.3 Torque Balancing

Physical measurement revealed that the right-side motors (M1, M4) produce more torque than the left-side motors (M2, M3), likely due to manufacturing variability in the low-cost motors. Without compensation, this imbalance causes the robot to veer during straight-line motion, degrading dead-reckoning accuracy.

The firmware applies a fixed **duty reduction of** -3.0 to the right-side motors, with a lower clamp of $0.5\times$ `DRIVE_CLAMP_LOW` to prevent the reduction from stopping a motor entirely. This simple calibration corrects the systematic drift.

7.4 Kickstart Mechanism

Low-cost brushed motors have high static friction relative to kinetic friction. When a motor transitions from stationary to moving, the initial PWM duty may be insufficient to break static hold, causing a brief stall. The **Kickstart** mechanism addresses this by applying an elevated duty (nominally 70) for **3 control frames** (3 ms at 1 kHz) whenever a motor abruptly transitions from zero speed to nonzero speed. The boost is proportional to the target speed to prevent overshoot during slow manoeuvres.

7.5 Operating Modes

The motor controller dynamically selects among three operating modes based on the robot's proximity to its target:

7.5.1 Normal Mode (> 50 mm from target)

Standard mecanum control with kickstart, dead zone compensation, and torque balancing. The duty floor is `DRIVE_CLAMP_LOW` = 65.

7.5.2 Precision Mode (< 50 mm from target)

When within 50 mm of the target, the robot enters **Precision Mode** with a specialized **Induction Kickstart**. Unlike the standard kickstart (which always fires), the induction kickstart is *conditional*: it activates only when the motors are stalled—defined as commanded speed > 0.05 while observed speed < 1.5 mm/s. If the robot is already moving, the boost is suppressed to prevent the oscillatory "wiggle" that occurs when repeated kickstarts fight the precision corrections.

The **Settling Band** (< 20 mm) uses a linear ramp-to-zero duty rather than a fixed floor, allowing the motors to produce arbitrarily small forces for the final approach.

7.5.3 Single-Motor Mode (< 15 mm from target, < 10 mm/s)

This is a novel operating mode designed for **extreme micro-adjustments**. When within 15 mm of the target and moving slowly, the firmware activates only *one* wheel at a time, producing the smallest possible robot displacement (∼1–2 mm per step).

The wheel selection uses a **dot-product alignment** algorithm:

1. Compute the unit velocity vector in the desired direction: $\hat{\mathbf{d}} = (d_x, d_y)$.
2. For each mecanum wheel i, compute the dot product of $\hat{\mathbf{d}}$ with the wheel's velocity contribution vector $\hat{\mathbf{w}}_i$.
3. Select the wheel with the largest positive dot product (most aligned with the desired direction).
4. Drive that single wheel at a conservative fixed duty (`DRIVE_CLAMP_LOW` + 5 = 70).

The wheel velocity vectors for the standard mecanum layout are:

$$\hat{\mathbf{w}}_1 = (-1, -1)/\sqrt{2} \qquad \text{(Front Right)} \tag{14}$$
$$\hat{\mathbf{w}}_2 = (-1, +1)/\sqrt{2} \qquad \text{(Back Right)} \tag{15}$$
$$\hat{\mathbf{w}}_3 = (+1, +1)/\sqrt{2} \qquad \text{(Back Left)} \tag{16}$$
$$\hat{\mathbf{w}}_4 = (+1, -1)/\sqrt{2} \qquad \text{(Front Left)} \tag{17}$$

By firing only the most-aligned wheel, the robot avoids introducing parasitic rotation (which all four-wheel actuation inevitably creates due to imperfect motor matching) and achieves displacements far below what the standard mecanum formula can reliably produce.

7.6 Anti-Slip and Stall Detection

Despite the traction analysis presented in **??**, wheel slip can still occur on contaminated surfaces or when the robot's weight shifts during servo actuation. The firmware implements a **stall detector** that monitors the discrepancy between commanded and observed velocity:

- **Trigger condition:** Observed velocity $< 5\,\text{mm/s}$ while commanded speed $> 20\,\text{mm/s}$ for **75 consecutive control ticks** (75 ms).
- **Response:** Apply a $1.35\times$ multiplicative duty boost to all motors for up to 100 ticks.

The detection requires sustained stall (75 ticks) rather than instantaneous detection to avoid false triggers during normal deceleration, where the commanded speed briefly exceeds the observed speed as the S-curve ramps down.

7.7 Active Braking (Reverse Thrust)

When a motion segment ends, physical momentum causes the robot to coast past the target. The firmware applies **active reverse thrust** to kill this momentum:

- **Rotation braking:** Reverse duty 75 for 15 ms (15 control ticks), triggered when the prior motion was rotation-only and the angular velocity exceeded 5 deg/s.
- **Translation braking:** Reverse duty 80 for 18 ms (18 control ticks), triggered when the observed speed exceeds 5 mm/s. The reverse thrust direction is computed from the Kalman-estimated velocity vector, projected into the robot's heading frame.

Translation braking is **suppressed during HOLDING state** to prevent oscillation: the HOLDING corrections are intentionally small, and applying reverse thrust after each one would cause the robot to oscillate back and forth around the target.

7.8 Drift Trim

Empirical testing revealed a systematic lateral drift during forward motion, attributed to manufacturing asymmetries in the mecanum rollers. The firmware pre-rotates the (V, H) velocity vector by a calibrated **drift trim angle** before applying it to the motors. The trim is speed-dependent:

Table 7: Drift trim calibration values (Robot A).

Speed Level	Trim Angle
SLOW	9.36ř
NORMAL	0.00ř
FAST	0.00ř

7.9 Motor Duty Normalization

If the combined effects of translation, rotation, and feedforward produce motor duties exceeding the hardware maximum (`DRIVE_CLAMP_HIGH` = 100), all four motor duties are proportionally scaled down to fit within the limit. This preserves the intended velocity direction and rotation ratio while preventing saturation.

8 Dead Reckoning and Latency-Compensated Vision

8.1 Dead Reckoning Engine

The `DeadReckoning` subsystem integrates commanded velocities at each 1 ms control loop tick to maintain a continuous position estimate. It employs a **two-tier position model**:

1. **Internal Odometer** (ix, iy, ia): Continuously integrated from velocity. Never resets except at startup.
2. **Global Anchor** (gx, gy, ga): Absolute correction from camera observations. Set by the latency compensator.

The current best-estimate position is computed as:

$$\mathbf{p}_{\text{current}} = \mathbf{p}_{\text{anchor}} + (\mathbf{p}_{\text{odometer}} - \mathbf{p}_{\text{odometer@anchor}}) \tag{18}$$

This model separates the accumulated error (captured in the anchor offset) from the incremental tracking (which remains accurate over short intervals), enabling smooth camera corrections without disrupting the odometric continuity.

8.1.1 Distance Calibration

Dead reckoning accuracy depends critically on the mapping between commanded velocity and actual displacement. The firmware applies empirically calibrated **distance factors**: 1.0× for vertical (forward/backward) and 1.25× for horizontal (strafe) motion on Robot A. The horizontal factor reflects the $\sim$ 20% systematic under-travel inherent to mecanum strafing, where roller slip dissipates a fraction of the commanded displacement.

8.1.2 EMA-Smoothed Velocity for Integration

A subtle but important detail: the dead reckoning uses an **EMA-smoothed** commanded velocity rather than the instantaneous value for integration. When the motion queue's speed scale drops to zero (segment end), the raw velocity falls to zero instantly, but the robot physically coasts for ∼30 ms. The EMA velocity decays naturally over this period, keeping the DR position estimate accurate through the coasting phase.

8.1.3 Position History Ring Buffer

The engine maintains a **ring buffer of 256 position snapshots**, each recording the internal odometer state and timestamp. This buffer is protected by a dedicated FreeRTOS mutex (separate from the main state mutex) to allow Core 0 to query historical positions during camera update processing without blocking Core 1's control loop. The latency compensator uses this buffer to look up "where did the robot think it was at the moment the camera frame was captured?"

8.2 Kalman-Filtered Velocity Estimation

Instantaneous velocity computed from position differences is noisy, especially at 1 kHz where quantization effects dominate. The motion queue employs a **1D Kalman filter per axis** (X and Y independently) with a position-velocity state model:

$$\mathbf{x}_k = \begin{bmatrix} p \\ v \end{bmatrix}, \quad \mathbf{F} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 \end{bmatrix} \tag{19}$$

The filter parameters are:

Table 8: Kalman filter noise parameters.

Parameter	Symbol	Value
Process noise (position)	q_p	0.5
Process noise (velocity)	q_v	50.0
Measurement noise	r	2.0

The relatively high process noise for velocity ($q_v = 50.0$) allows the filter to track rapid velocity changes (as commanded by the S-curve profiler) while still rejecting noise.

A **Stationary Deadzone** of 5.0 mm/s is applied to the Kalman velocity estimate: if the estimated speed falls below this threshold, it is clamped to zero. This prevents stale velocity signals from influencing the braking logic when the robot is nominally stationary but the Kalman filter has not yet fully converged to zero.

8.3 Clock Synchronization

The robot and phone operate on independent clocks with different origins and potential drift. The system achieves millisecond-level synchronization through a **PING/PONG protocol**:

1. The robot sends `P:{robot_timestamp}` every 500 ms.

2. The phone replies with `Q:{robot_timestamp}:{phone_timestamp}`.

3. The robot computes the round-trip time (RTT) using an exponential moving average ($\alpha = 1/8$) and derives the clock offset: $\Delta_{\text{clock}} = t_{\text{phone}} - t_{\text{robot}} - \text{RTT}/2$.

This clock offset is essential for the latency compensator to correctly timestamp camera observations in the robot's time domain.

8.4 Latency-Compensated Vision

The smartphone camera introduces an inherent ∼150 ms latency between frame capture and the arrival of the processed position observation at the robot. Naïvely applying this observation as a direct position correction would cause the robot to "jump back" to where it was 150 ms ago, creating oscillatory behavior.

The **Latency Compensator** solves this with a retroactive correction algorithm:

1. When a camera update `C:{ts}:{x}:{y}:{angle}` arrives, the compensator converts the phone timestamp t_{phone} to robot time using the synchronized clock offset, and subtracts the assumed camera processing latency (150 ms).

2. It queries the dead-reckoning history buffer (**??**) to find the odometric state at the *capture instant*: "Where did the robot think it was when this frame was taken?"

3. The difference between the camera observation and the historical position estimate is the **measured drift**:

$$\mathbf{e}_{\text{drift}} = \mathbf{p}_{\text{camera}} - \mathbf{p}_{\text{DR@capture}} \tag{20}$$

4. If $\|\mathbf{e}_{\text{drift}}\| > 5.0$ mm (the drift threshold), a correction is triggered. The drift error is blended forward into the current position estimate over a 200 ms interpolation window using a smooth correction policy, preventing jarring jumps.

5. If $\|\mathbf{e}_{\text{drift}}\| > 25.0$ mm (the emergency threshold), an immediate correction is applied to prevent the robot from operating on a grossly incorrect position belief.

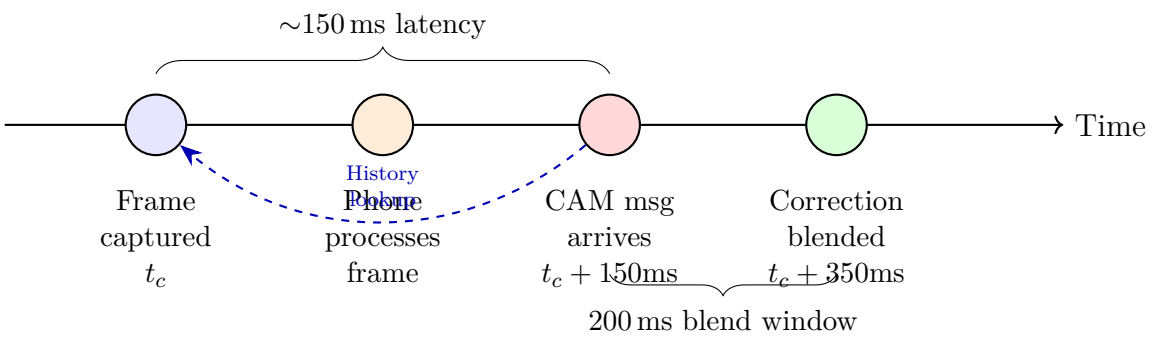

Figure 4: Latency compensation timeline. When a camera observation arrives at $t_c + 150$ms, the compensator looks up the dead-reckoning state at t_c (the actual capture time), computes the drift error, and blends the correction over a 200 ms window to prevent position jumps.

8.5 Heading Authority

For heading stabilization during translation, the controller must choose between the dead-reckoning angle (high frequency, but drifts) and the camera angle (accurate, but low frequency and prone to jumps). As described in **??**, the **Blended Angle**—an EMA fusion of both sources—provides the best of both worlds. This blended angle serves as the *heading authority* for:

- The PID heading stabilization loop (computing heading error)
- The heading-frame velocity transform (projecting world-frame velocities into the robot frame)
- The feedforward projection (transforming world-frame acceleration into the robot frame)

9 Android Controller Application

The Android companion application (`PineappleRobotPro`) serves three roles: external vision system, mission controller, and real-time telemetry display. It is implemented in Kotlin with Jetpack Compose for the UI layer.

9.1 Architecture Overview

The application is decomposed into the following components:

Table 9: Android application component architecture.

Component	Responsibility	Size
`MainActivity`	UI composition, state management, camera frame processing, path editor	75 kB
`UIComponents`	Reusable Compose UI elements: field view, robot overlays, control panels	98 kB
`RobotUdpBridge`	UDP communication, robot discovery, tap-target and heading-lock control loops	31 kB
`CameraComponents`	CameraX integration, perspective warp, ArUco detection via OpenCV	15 kB
`ContinuousMotionController`	Multi-waypoint path sequencer, servo action scheduling	11 kB
`DataModels`	Protocol enums, path graph data structures, field constants	5 kB
`StorageHelpers`	Configuration persistence, path serialization	22 kB
`NavigationHistoryManager`	Undo/redo for path editing operations	2 kB

9.2 Vision Pipeline

The vision pipeline transforms raw camera frames into robot position observations:

1. **Frame Capture:** CameraX provides frames at the device's native rate (typically 30 Hz) in RGBA format. The app preferentially selects the **ultra-wide lens** (focal length < 2.5 mm) to maximize the field of view.

2. **Perspective Warp:** Four user-calibrated corner points define the playing field boundary. A perspective transform (`Imgproc.getPerspectiveTransform`) warps the captured frame into a rectified top-down view of the field. The transform accounts for:

   - A configurable padding region around the field boundary to track robots that have moved slightly outside the playing area
   - The field's aspect ratio (1143 mm × 1181 mm)

3. **ArUco Detection:** The warped, grayscale image is processed by OpenCV's `ArucoDetector` using a 4×4 dictionary (50 markers). Each robot carries a unique ArUco marker (ID 0 for Robot A, ID 1 for Robot B).

4. **Pose Extraction:** For each detected marker, the center position is computed as the mean of the four corner coordinates (normalized to the field boundary), and the heading angle is extracted from the vector between the top and bottom edge midpoints:

$$\theta = \operatorname{atan2}(-\Delta_y,\ \Delta_x) \cdot \frac{180^\circ}{\pi} \tag{21}$$

   The use of ArUco marker detection is a standard technique and is not claimed as a contribution; rather, it serves as a reliable, zero-cost fiducial system that integrates well with the OpenCV library already required for perspective warping.

5. **Coordinate Transform:** Normalized marker positions are scaled to millimetres using the known field dimensions and transmitted to the robot as `CAM` messages with the capture timestamp.

9.3 Robot Discovery and Connection

The `RobotUdpBridge` implements an **automatic discovery protocol**:

1. The app listens on UDP port 4210 for incoming packets.

2. When a robot sends a `HELLO` or `REGISTER` message, the bridge records the robot's ID, IP address, and last-seen timestamp.

3. Robots are considered connected if their last-seen time is within 10 s.

4. All subsequent commands are routed to the robot by ID, with the IP address resolved from the registry.

This design supports **multi-robot operation**: multiple robots can connect simultaneously, each identified by its unique ID ("A", "B"). The phone broadcasts camera observations to all visible robots and routes commands per-robot.

9.4 Tap-Target Navigation

The app implements a **Tap-Target** control mode where the user taps a position on the field view, and the app autonomously navigates the robot to that location. The tap-target controller operates as a periodic feedback loop:

1. On each camera update, the controller checks if the robot is within the arrival tolerance (X: 10 mm, Y: 15 mm, angle: 4°).
2. If outside tolerance, it sends a `WAYPOINT` command to the robot at a rate governed by a **distance-based cooldown**:
   - > 150 mm: 100 ms cooldown (fast updates)
   - 80–150 mm: 160 ms
   - 40–80 mm: 250 ms
   - < 40 mm: 400 ms (precision pulsing)
3. Arrival is confirmed only after **3 consecutive stable frames** within tolerance, preventing premature declaration when the robot passes through the target while overshooting.

The asymmetric X/Y tolerances (10 mm vs. 15 mm) account for the inherent difficulty of mecanum control in the forward/backward axis versus strafe.

9.5 Path Sequencer (Continuous Motion Controller)

For autonomous mission execution, the app implements a **Continuous Motion Controller** that sequences multi-waypoint paths:

1. A course is defined as an ordered list of `Waypoint` objects, each specifying a position, heading, tolerances, and optional servo actions.
2. The sequencer navigates to each waypoint in order by delegating to the tap-target system.
3. Upon arrival at each waypoint, the sequencer executes any associated servo actions sequentially, waiting 800 ms between actions.
4. After all servo actions complete, the sequencer advances to the next waypoint.
5. The sequencer supports **cross-robot wait conditions**: a waypoint can specify that it should not be started until a different robot has reached a specific node, enabling coordinated multi-robot missions.

9.6 Graph-Based Path Editor

The app includes a visual path editor that allows users to design robot courses directly on the warped field view:

- **Node Placement:** Tap to place nodes at specific field coordinates.

- **Connection Management:** Draw connections between nodes to define traversal order.

- **Per-Node Configuration:** Set rotation angle, servo actions (two slots per node), tolerances, and cross-robot wait conditions.

- **Fruit Detection Integration:** Nodes can reference specific fruit positions from a hard-coded grid, with the app automatically determining the appropriate servo action based on detected fruit color (red, green, or absent).

- **Undo/Redo:** Full undo/redo support via the `NavigationHistoryManager`.

- **Persistence:** Paths are serialized and persisted to device storage, surviving app restarts.

10 Integrated Control Flow

The complete control flow from camera observation to motor actuation spans both platforms and multiple algorithmic stages. **??** illustrates the end-to-end pipeline.

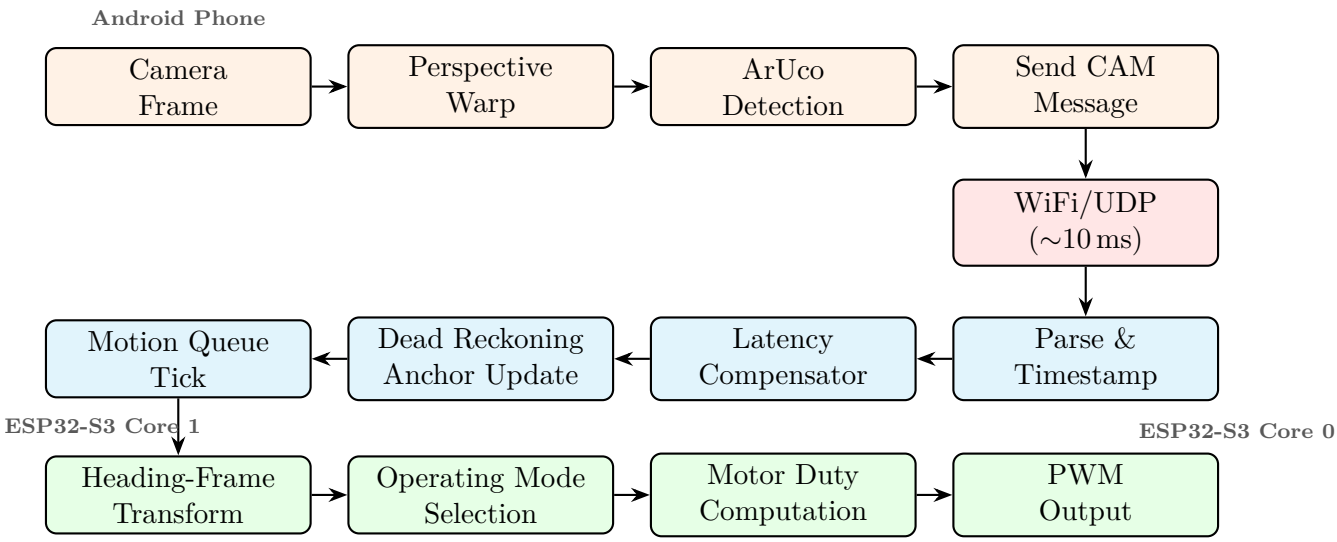

Figure 5: End-to-end control flow from camera capture to motor actuation. The pipeline spans the phone (camera processing), the network (UDP), Core 0 (parsing), and Core 1 (motion control).

The complete pipeline latency from frame capture to motor response is approximately:

Table 10: Pipeline latency breakdown.

Stage	**Latency**
Camera capture + CameraX pipeline	∼33 ms (at 30 fps)
Perspective warp + ArUco detection	∼30–50 ms
UDP network transit	∼5–20 ms
Firmware parsing + latency comp.	∼1 ms
Control loop tick	$\leq$ 1 ms
Motor PWM update	$<$ 0.1 ms
Total (estimated)	∼70–105 ms
Assumed for compensation	150 ms (conservative)

11 S-Curve Tuning Strategy Across Robot Configurations

The physical measurements in **??** reveal a critical constraint: Robot B's traction limit (3652 mm/s^2) is less than half of Robot A's (8308 mm/s^2). Since the firmware must produce reliable dead reckoning on both robots without risking wheel slip, the S-curve profiler's operational bounds must be capped below the **lowest common traction limit** across the fleet.

The current S-curve parameters (peak acceleration 250 mm/s^2, peak jerk 1200 mm/s^3) are set conservatively below Robot B's traction limit by a factor of $\sim 14.6\times$. This generous margin accounts for:

- Surface variability (the competition field may have different friction characteristics than the calibration surface)
- Dynamic weight shifts during servo actuation
- The cumulative effect of feedforward compensation, which adds to the commanded acceleration
- The asymmetric deceleration limits (2× acceleration), which bring the peak deceleration to 500 mm/s^2—still well within the traction envelope

12 Speed Profiles and Operational Parameters

The firmware supports three speed levels, each calibrated from PS2 controller physics at 100% duty:

Table 11: Speed profile calibration values (default, before scaling).

Speed Level	**Translation**	**Rotation**
SLOW	29.0 mm/s	15.0 deg/s
NORMAL	43.5 mm/s	51.84 deg/s
FAST	72.5 mm/s	86.4 deg/s

These values may appear modest compared to larger competition robots, reflecting the deliberately low-cost, low-power drive system. The system compensates for limited speed with high precision: the combination of S-curve profiling, predictive braking, and single-motor micro-adjustments enables the robot to achieve its 5 mm waypoint tolerance consistently despite peak speeds of only 72.5 mm/s.

12.1 Precision Thresholds

Table 12: Precision control thresholds.

Parameter	**Value**
Precision mode activation distance	50 mm
Single-motor mode activation	15 mm
Waypoint arrival tolerance	5 mm
Rotation arrival tolerance	1.5°
Close approach distance	120 mm
Waypoint deduplication tolerance	15 mm
Heading stabilization gain	2.5
Maximum stabilization ω	35 deg/s

13 Discussion

13.1 Novel Contributions

The Pineapple-Bot-Pro system makes several technical contributions within the context of low-cost educational robotics:

1. **Retroactive Latency Compensation:** Rather than trying to reduce the camera pipeline latency (which is fundamentally limited by smartphone hardware and processing), the system embraces the latency and *compensates retroactively* using the dead-reckoning history buffer. This approach is applicable to any system where observation latency is known and deterministic localization is available between observations.

2. **Jerk-Limited S-Curve on a $3 Microcontroller:** S-curve trajectory profiling is typically associated with industrial motion controllers. Implementing it in a 1 kHz control loop on an ESP32-S3 demonstrates that the computational requirements are well within the reach of modern low-cost microcontrollers.

3. **Single-Motor Micro-Adjustment:** The dot-product wheel selection algorithm for sub-2 mm position corrections exploits the specific geometry of mecanum drives in a way not commonly described in the educational robotics literature.

4. **Blended Angle Stabilization:** The EMA fusion of dead-reckoning and camera angles for heading authority provides a smooth, accurate reference without the complexity of a full sensor fusion framework (e.g., EKF/UKF).

5. **Phone-as-Vision-Sensor:** Using a consumer smartphone as both the external vision system and the mission controller eliminates the need for dedicated vision hardware, reducing system cost and setup complexity.

13.2 Limitations and Future Work

- **No Encoder Feedback:** The current system relies entirely on open-loop dead reckoning (time × speed integration) without wheel encoders. Adding encoders would significantly improve DR accuracy and enable closed-loop motor speed control.
- **WiFi Reliability:** The WiFi link is susceptible to interference in crowded competition venues. A dedicated radio link (e.g., ESP-NOW, which uses the same ESP32 hardware) could reduce latency and improve reliability.
- **Camera Frame Rate:** The smartphone camera operates at 30 fps, limiting the vision update rate to ∼30 Hz. Higher frame rates would improve the blended angle convergence and reduce the reliance on dead reckoning between observations.
- **Single-Axis Kalman Filters:** The per-axis Kalman filters do not model cross-axis coupling. A 4-state coupled filter (x, y, v_x, v_y) could improve velocity estimation during diagonal motion.

14 Conclusion

This report has presented the Pineapple-Bot-Pro, a low-cost autonomous mecanum-drive robot system designed for the Vietnam Robotics For Good national competition. The system demonstrates that sophisticated autonomous behavior—including jerk-limited trajectory profiling, latency-compensated vision, single-motor micro-adjustments, and multi-robot coordination—can be achieved on commodity hardware costing well under $50 USD per robot.

The key insight driving the design is that modern low-cost microcontrollers (such as the ESP32-S3) have sufficient computational power to run advanced control algorithms at high frequencies, making software sophistication the most cost-effective path to improved robot performance. The dual-core architecture, running a 1 kHz control loop alongside asynchronous network processing, enables deterministic motor control while maintaining responsive communication—a capability that would have required dedicated hardware a decade ago.

The system achieves ±5 mm positional accuracy and ±1.5° angular accuracy on a 1143 mm × 1181 mm competition field, using only a consumer smartphone for external localization. These results validate the design philosophy of investing in algorithmic complexity rather than expensive hardware, and suggest that similar approaches could benefit other resource-constrained robotics applications.

References

[1] Espressif Systems, "ESP32-S3 Technical Reference Manual," Version 1.1, 2022. [Online]. Available: https://www.espressif.com/en/products/socs/esp32-s3

[2] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, "Automatic generation and detection of highly reliable fiducial markers under occlusion," *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014.

[3] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA: MIT Press, 2011.

[4] G. Welch and G. Bishop, "An Introduction to the Kalman Filter," University of North Carolina, Chapel Hill, TR 95-041, 2006.

[5] Tower Pro, "SG90 Micro Servo Datasheet," [Online]. Available: http://www.towerpro.com.tw/product/sg90-7/

[6] PlatformIO, "PlatformIO: An open source ecosystem for IoT development," [Online]. Available: https://platformio.org/

[7] OpenCV, "ArUco Marker Detection," OpenCV Documentation. [Online]. Available: https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html

[8] R. A. Brooks, "A robust layered control system for a mobile robot," *IEEE Journal on Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986.

A Configuration Defaults Reference

Table 13: Complete firmware configuration parameters with default values.

Parameter	Default	Unit
Motor Physics		
DRIVE_CLAMP_LOW	65	duty
DRIVE_CLAMP_HIGH	100	duty
DRIVE_CLAMP_FINE	65	duty
MOTOR_MAP_SLOPE	43.017	—
MOTOR_MAP_OFFSET	57.165	—
KICKSTART_FRAMES	3	ticks
KICKSTART_SPEED	70.0	duty
RAMP_DURATION_MS	75	ms
RAMP_START_FRACTION	0.15	—
Precision Control		
PRECISION_MODE_THRESH_MM	50	mm
WAYPOINT_TOLERANCE_MM	5.0	mm
ROTATION_TOLERANCE_DEG	1.5	deg
CLOSE_APPROACH_DISTANCE_MM	120	mm
STABILIZATION_GAIN	2.5	—
MAX_STABILIZATION_OMEGA	35.0	deg/s
Trajectory Planning		
SCURVE_MAX_ACCEL_MM_S2	250	mm/s^2
SCURVE_MAX_JERK_MM_S3	1200	mm/s^3
FEEDFORWARD_KV	0.02	—
FEEDFORWARD_KA	0.04	—
ADAPTIVE_LOOKAHEAD_BASE_S	0.06	s
ADAPTIVE_LOOKAHEAD_GAIN	0.0022	—
Braking		
PREDICTIVE_BRAKE_DECEL	350	mm/s^2
PREDICTIVE_BRAKE_SAFETY	1.2	—
LATENCY_AWARE_DECEL	300	mm/s^2
ROTATION_BRAKE_FRAMES	15	ticks
ROTATION_BRAKE_DUTY	75	duty
TRANSLATION_BRAKE_FRAMES	18	ticks
TRANSLATION_BRAKE_DUTY	80	duty
Vision & DR		
CAMERA_LATENCY_MS	150	ms
DRIFT_THRESHOLD_MM	5.0	mm
EMERGENCY_THRESHOLD_MM	25.0	mm
CORRECTION_BLEND_MS	200	ms
KALMAN_PROCESS_NOISE_POS	0.5	—
KALMAN_PROCESS_NOISE_VEL	50.0	—
Anti-Slip Detection		
SLIP_CMD_SPEED_THRESH	20.0	mm/s
SLIP_OBS_SPEED_THRESH	5.0	mm/s
SLIP_DETECT_TICKS	75	ticks
SLIP_BOOST_FACTOR	1.35	—
Timing		
CONTROL_LOOP_INTERVAL_MS	1	ms
STATUS_INTERVAL_MS	100	ms
HELLO_INTERVAL_MS	2000	ms

B UDP Protocol Message Formats

```
Robot → Phone:
  H:<robot_id>:<ip_address>                HELLO (3s interval)
  R:<robot_id>:mecanum4wd:<ip>              REGISTER (on boot)
  S:<y>:<x>:<angle>:<qLen>:<drift>          STATUS (100ms interval)
  P:<timestamp_ms>                          PING (500ms interval)
  L:S:<message>                             Log (static)
  L:U:<id>:<message>                        Log (update-in-place)
  L:I:<message>                             Log (important/red)

Phone → Robot:
  M:<dir>:<dist_mm>:<speed>:<policy>[:action]        MOVE
  D:<dir>:<dur_ms>:<speed>:<policy>                  MOVE_DURATION
  W:<x>:<y>[:<angle>]:<speed>:<policy>[:action]      WAYPOINT
  T:<target_angle>:<speed>:<policy>                  ROTATE
  O:<cw|ccw>:<dur_ms>:<speed>:<policy>               ROTATE_DURATION
  V:<vx>:<vy>:<omega>:<timeout_ms>                   VELOCITY
  C:<timestamp>:<x_mm>:<y_mm>:<angle_deg>            CAM
  E:<action_code>                                    SERVO_EXEC
  U:<0|1>                                            SERIAL_MONITOR
  Q:<orig_ts>:<phone_ts>                             PONG
  X                                                  DONE
  A                                                  ABORT
```

C Servo Action Reference

Table 14: Supported servo actions and their protocol codes.

Code	Action	Sequence
Robot A (Fruit Picker)		
LL	LOWER_LEFT	Grabber left → wait 200ms → center
LR	LOWER_RIGHT	Grabber right → wait 200ms → center
UL	UPPER_LEFT	Slider down → 200ms → arm down → 100ms → slider up + arm up
UR	UPPER_RIGHT	Slider up → 100ms → arm down → 100ms → arm up
GL/GR/GC	Grabber primitives	Direct servo position
SU/SD	Slider primitives	Direct servo position
AD/AU	Arm primitives	Direct servo position
Robot B (Dispenser)		
DO/DC	Dropper open/close	Direct servo position
CO/CC/CW	Ceiling open/close/water	Direct servo position